Static Analysis for Bounds Check Elision in the Naga WGSL Compiler

Bhuvan B ACM SIGPLAN-M Mentee | Open Source Contributor

IICT 2026 · Paper #55

Proving array indices in bounds removes 102 of the 186 bounds clamps in vello's GPU shaders.

vello's GPU work then runs **7.0% faster on an Intel iGPU**, but only **0.6% faster on an NVIDIA GPU**.

Suite geomean over 32 vello test scenes per GPU; 95% CI +3.7 to +10.4% (Intel), +0.2 to +1.0% (NVIDIA). Recorded vello GPU work replayed with checks on, as a browser must run it.

code, data and the source of every number

How: one proof, and a tree reduction loses all four clamps

```
@workgroup_size(256) 1
var<workgroup> s: array<u32, 256>;

for (var stride = 128u; stride > 0u;
     stride = stride >> 1u) { 2
  if (lid < stride) { 3
    s[lid] = s[lid] + s[lid + stride]; 4
  }
  workgroupBarrier();
}
```

1. `lid` ∈ [0, 255]: the workgroup size
2. `stride` ∈ [1, 128]: its only update is `>> 1`
3. `lid < stride` ⇒ `lid` ∈ [0, 127]
4. `lid + stride` ∈ [1, 255] fits `s` ✓

SPIR-V clamps (UMin)

~~`UMin %lid %uint_255`~~
~~`UMin %lid %uint_255`~~
~~`UMin %lid_plus_stride %uint_255`~~
~~`UMin %lid %uint_255`~~

4 → 0 clamps. The binary is byte-identical (sha256) to the build with all checks off.

Shown with wgpu's Vulkan settings; SPIR-V ids renamed for reading.

1 Today, wgpu clamps array indices

WGSL forbids out-of-bounds memory access. On Vulkan, Metal and D3D12, wgpu rewrites `s[i]` as **`s[min(i, 255)]`**.

	arrays & vectors	storage buffers
Vulkan	clamp	hardware (robustBufferAccess2)
Metal	clamp	clamp
D3D12	clamp	clamp
GLES	none	none

A browser must keep these checks. Native apps can turn them off (vello does).

2 How a fact is proved

WGSL → Naga IR → **validator: interval analysis** → SPIR-V / MSL / HLSL

Every integer carries an **interval**. Branches narrow it; a loop variable matches a known **recurrence** or becomes unknown. A proved access loses its clamp.

Branch guard

```
var i = data[g.x];
if (i < 8u) { … lut[i] … }
```

i ∈ [0, 7] in the branch ✓

Bit mask

```
lut[data[g.x] & 7u]
```

x & 7 ∈ [0, 7] ✓

Loop recurrence

```
for (var k = 128u; k > 0u;
     k = k >> 1u)
```

k ∈ [1, 128] in the body ✓

Relation a ≥ b

```
if (lid >= off) {
  t = s[lid - off]; }
```

lid − off ∈ [0, 254] ✓

Not proved, so the clamp stays: runtime-sized arrays, bounds like `i < arrayLength(&a)`, facts across calls, guards on `let` values.

3 Why trust a removed check?

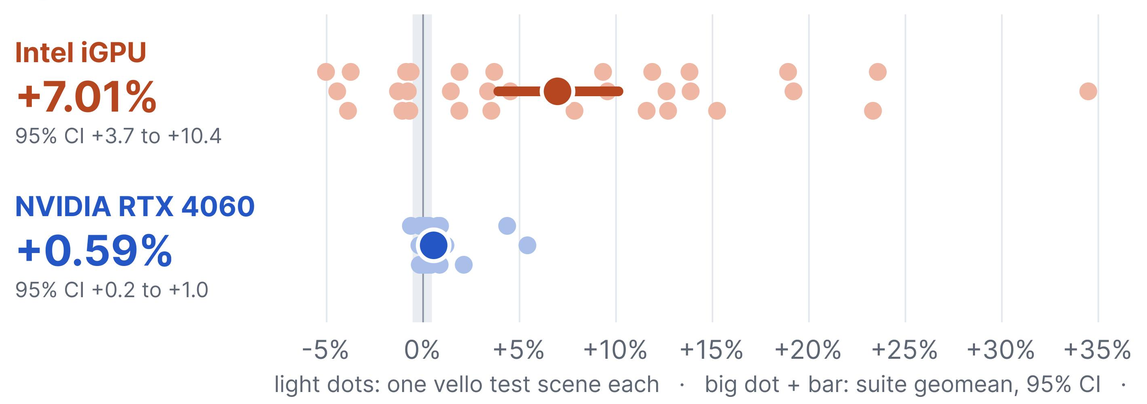
Tests: 95 hand-written shaders where a check must stay or must go; interval arithmetic checked against exhaustive enumeration.

On GPU: 8 of 8 differential runs give byte-identical output with and without elision.

Audit: we found 3 unsound loop/branch shapes and fixed each with a regression test. No measured binary changed.

Not yet covered: shapes beyond these tests, and `discard` on Metal/D3D12. Next: random-program differential testing.

4 Real code: vello as web content



32 scenes per GPU, checks kept vs. elided, against an identical-copy control; pre-registered. Intel: 22 faster, 10 slower. NVIDIA: 22 faster, 4 slower, 6 unchanged.

Synthetic kernels: in 6 hand-written GPU kernels all 29 clamps go; measured speedups range from none detectable to +15.8%.

5 Coverage in 9 open-source projects

runtime-indexed accesses
729

into fixed-size arrays or vectors: provable in principle
342

proved in bounds today
118

364 (49.9%) index runtime-sized arrays: the next step is symbolic lengths.

6 Compared with Tint (Chrome)

	this work	Tint
facts from any if guard	✓ yes	× loop exits only
relation a ≥ b for a − b	✓ yes	× no
shrinking / geometric loops	✓ yes	× ±1 steps only
runtime-sized arrays	× no	× no
shipped	× no (fork)	✓ default-on

From reading Tint's IntegerRangeAnalysis source, not measured. Classic CPU work: ABCD, Go `prove`, LLVM IRCE.